This Group CA covers building a complete data model for an application, including repository and data access layers. Each group is made up of 3 team members.

You have been tasked with writing a music library system. Your system should provide the following functionality:

1) User logic:
    a. Register with the library. Note: You should implement and document a **reasonable** password policy.
    b. Log in to your account (by providing your username and password).
    c. Change your password
2) General access:
    a. View all artists in the library.
    b. View all albums for an artist.
    c. View all genres.
    d. Search for songs by title/artist/album/genre.
3) Rating songs:
    a. Rate a song (1-5). Rating a song again replaces your previous rating.
    b. View all songs you have rated, including song details and your rating.
    c. Get top-rated (highest average) song. Where multiple songs are tied, all should be returned.
    d. Get all songs you have rated at a specific level (e.g. all songs you have rated a 2).
4) Log out.

You are required to:

1. Create a MySQL database to hold all the required information for the system to function in a well-designed fashion. Primary and foreign keys should be included, and the database should include reasonable levels of information, not merely the minimum required to allow the system to function.
2. Implement the **data access layer** for the system – this should control all access to your MySQL database and should adhere to the DAO pattern. Dependency injection (via constructors) should be employed to increase design flexibility. You should consider employing Java's `record` type to create the DTOs.
    a. Note: DAOs should correspond to tables within the database schema and should contain database actions **only** (e.g. password logic would **not** be found in a DAO).
3. Implement the **repository layer** for the system – this should be employed by the user interface and should use the appropriate DAOs to carry out any necessary database actions. Dependency injection (via constructors) should be employed to increase design flexibility.
    a. Note: Repositories should focus more on **use-cases** clustered based on logical area and may combine calls to multiple DAOs to produce their results. For example, catalogue-related actions may require information about songs, genres or albums all at once.
4. Implement the **front-end interface** for the system – this MUST be based on the command-line and must use the repository layer exclusively to interact with the database.
5. Write a set of Junit *integration* tests for **one** DAO & repository per student (DTOs do not need to be tested)
6. Write Javadoc comments for **all** methods. These should include information on return types, parameters, method functionality and the meaning of any potential exceptions that can be thrown.
    a. Note: Where applicable, both interface signatures & method implementations should be commented.

Project management:

You are **required** to maintain source control using git and github. Code should be regularly committed (with appropriate commit messages indicating what is being added/changed/removed/fixed etc) and pushed to the shared repository by all team members. Commit history may be used as evidence when assessing individual contribution. Students should develop in their own branches, with main always retained for a fully functional version of the project.

You are **required** to track your project management (using **Jira**) for the duration of this project, with a minimum of three sprints carried out, each lasting approx. one week. Issues and epics should be employed appropriately, and your backlog should include **all anticipated tasks/issues** weighted with story points and hours. Sprints should contain 4 phases/stages: To Do, In Progress, In Testing and Completed. Your SCRUM board should be updated **as the work occurs**.

Marks will be awarded as follows:

- Joint marking:
  - Database design and population of tables with dummy information – 10%
  - Program interface – 10%
- Individual marking:
  - Data access layer – 30%
  - Repository layer – 20%
  - Junit tests – 15%
  - Javadocs – 5%
  - Project management – 10%

Group work balance:

Each class's owner(s) should be listed using the @author Javadoc annotation at the top of the class. Where more than one student has worked on a class, they should be listed as an author. However, each student must take **full ownership** of **at least one** DAO and repository, including the associated interfaces, implementations, DTOs and integration tests.

Submission instructions:

Upload a link to your project on GitHub (please remember to add me as a collaborator – username: @mgraham-dkit) Your git repository **MUST** include files containing the SQL statements used to create and populate your MySQL "live" and test databases. If these files are not included, **your project will not be marked**.

Deadline: 23:59 on 30th October 2026.